Лабораторная работа 4 "Управление состоянием в React-приложении"

Лабораторная работа №4. Управление состоянием в React-приложении

Ветка репозитория: `24`

---

1. ЦЕЛЬ РАБОТЫ

Освоение управления локальным состоянием в React с использованием хуков `useState` и `useEffect`. Формирование навыков обработки побочных эффектов, асинхронных операций и динамического обновления интерфейса **для реализации клиентской части вашего курсового проекта**.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Функциональные компоненты React, JSX.
- Хук `useState` (инициализация, обновление состояния).
- Хук `useEffect` (массив зависимостей, очистка эффекта).
- Жизненный цикл функционального компонента.
- Работа с формами и событиями в React.
- Работа с `localStorage` (чтение, запись).

---

3. ЗАДАЧИ РАБОТЫ

- Создать React-приложение (рекомендуется Vite).
- Реализовать компонент управления списком элементов, **соответствующим предметной области вашего курсового проекта** (например, список задач, товаров, контактов, книг, фильмов и т.п.).

- Использовать `useState` для хранения массива данных и значения поля ввода.
- Использовать `useEffect` для сохранения списка в `localStorage` (при каждом изменении) и загрузки при монтировании.
- Реализовать фильтрацию или сортировку списка (в соответствии с логикой вашего проекта).
- Добавить дополнительные побочные эффекты (таймер автосохранения, изменение заголовка страницы и т.п. – на выбор).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Создание проекта

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
```

Часть 2. Компонент управления списком

- Создать компонент, соответствующий вашей предметной области (например, `TaskList.jsx`, `ProductList.jsx`, `ContactList.jsx` и т.п.).
- Использовать `useState` для:
  - массива элементов (начальные данные можно загрузить из `localStorage` или задать моковыми).
  - значения поля ввода (для добавления/редактирования).
  - состояния фильтра/сортировки (если требуется).
- Реализовать добавление, удаление элемента, редактирование (по необходимости).

Часть 3. Хранение в `localStorage`

- `useEffect` с зависимостью `[items]` — сохранять массив в `localStorage` (использовать `JSON.stringify`).
- `useEffect` с пустым массивом — при монтировании загружать данные из `localStorage` (если есть) или инициализировать начальными данными.

Часть 4. Дополнительные эффекты (по выбору)

- Изменение заголовка страницы `document.title` в зависимости от количества элементов.
- Таймер автосохранения с debounce (задержка 500 мс после последнего изменения).
- Имитация загрузки с сервера (задержка 1 секунда) при старте.
- Подсчёт статистики (общее количество, выполненных/невыполненных, сумма и т.п.).

Часть 5. Тестирование

- Запустить приложение: `npm run dev`.
- Проверить: добавление, удаление, редактирование, фильтрацию, сохранение состояния после перезагрузки страницы.
- Убедиться, что `localStorage` содержит корректные данные.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

- Титульный лист, цель работы.
- Краткое описание приложения (предметная область вашего курсового проекта, реализованная функциональность).
- Скриншоты интерфейса (главная страница, формы добавления/редактирования, результат фильтрации/сортировки).
- Листинг основного компонента (ключевые части) с комментариями.
- Демонстрация работы: добавление, удаление, сохранение после перезагрузки страницы (скриншоты до и после).
- Ответы на контрольные вопросы (16 вопросов).
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.

- Ссылка на репозиторий GitHub с кодом (ветка `24`).
- **Индивидуальное выполнение. Вариант Темы** определяется по **Генератору вариантов.**
- Код должен быть загружен в репозиторий с веткой `24`.

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (16 вопросов)

1. Для чего нужен хук `useState`? Как обновить состояние на основе предыдущего значения?
2. Что такое побочный эффект (side effect)? Приведите примеры из вашего проекта.
3. Как работает массив зависимостей в `useEffect`? Что произойдёт, если оставить его пустым?
4. Как загрузить данные из `localStorage` при монтировании компонента?
5. Почему нельзя вызывать хуки внутри условий или циклов?
6. В чём разница между `localStorage` и `sessionStorage`?
7. Как реализовать «ленивую загрузку» начального состояния из `localStorage` в `useState` (функция-инициализатор)?
8. Как отладить бесконечный цикл перерендеров, вызванный `useEffect`?
9. Как сохранить объект или массив в `localStorage`? Как восстановить его обратно?
10. Что произойдёт, если изменить состояние сразу после вызова `setState`? Будет ли компонент перерендерен мгновенно?
11. Как имитировать асинхронную загрузку данных с сервера с помощью `setTimeout` в `useEffect`?
12. Какую функцию очистки (cleanup) можно вернуть из `useEffect`? Приведите пример отмены таймера из вашего проекта.
13. Как правильно работать с формами в React – контролируемые и неконтролируемые компоненты?
14. Как реализовать «горячую клавишу» (например, Enter для добавления элемента) в React?

15. Как можно оптимизировать производительность приложения с большим списком (виртуализация, `useMemo`, `React.memo`)?
16. Как в React реализовать debounce (задержку) для автосохранения при вводе текста? Напишите пример.

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Данная лабораторная работа является **клиентской и полностью автономной** (использует `localStorage` вместо серверного API). Однако в следующей лабораторной работе (№5) вы замените `localStorage` на взаимодействие с вашим REST API, добавив аутентификацию и CRUD-операции с сервером.
- Рекомендуется использовать компонентную архитектуру и выносить логику работы с `localStorage` в отдельный пользовательский хук (например, `useLocalStorage`). Это облегчит последующий рефакторинг.
- Обязательно обработайте случаи отсутствия данных в `localStorage` (инициализация начальными данными – например, парой примеров, соответствующих вашей предметной области).

---

9. КРИТЕРИИ ОЦЕНКИ

- Реализация базовых CRUD-операций (добавление, удаление, редактирование) в соответствии с предметной областью курсового проекта.
- Корректное использование `useState` и `useEffect`.
- Сохранение/загрузка данных в `localStorage`.
- Реализация фильтрации или сортировки списка.
- Наличие дополнительного побочного эффекта (по выбору).
- Оформление отчёта и ответы на контрольные вопросы.
- Ссылка на репозиторий с веткой `24`.